МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 8

Тема: Поведенческие паттерны: Посредник, снимок

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы поведенческих паттернов проектирования «Посредник» (Mediator) и «Снимок» (Memento) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два поведенческих паттерна из списка: «Посредник», «Снимок».

В соответствии с заданием были выбраны паттерны «Посредник» (Mediator) и «Снимок» (Memento).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу поведенческих паттернов: посредник координирует работу всех UI-компонентов, а снимок позволяет откатывать состояние приложения к предыдущим шагам.

1. Описание паттерна «Посредник»

Паттерн «Посредник» (Mediator) - это поведенческий паттерн проектирования, который позволяет уменьшить связанность множества классов между собой, заставляя их общаться через единый объект-посредник вместо прямых ссылок друг на друга.

Данный паттерн применяется в ситуациях, когда между компонентами системы много взаимных зависимостей, и прямое взаимодействие делает систему сложной для понимания и изменения.

В разработанном приложении паттерн «Посредник» используется для организации взаимодействия UI-компонентов. В программе определены следующие роли:

- UIMediator - посредник, координирующий работу всех компонентов;
- GridComponent - сетка ячеек (знает только о посреднике);
- RoundedButton - кнопки (не знают о других компонентах);
- tk.Label - метки (не знают о других компонентах).

Каждый компонент при событии вызывает:

self.mediator.notify(self, "event", data)

Посредник решает, что делать и какие компоненты обновить.

Обрабатываемые события:

- cell_clicked - клик по ячейке;
- mode_changed - переключение режима;
- clear - очистка последовательности;
- primary_action - основное действие (вход или создание);
- undo - откат к предыдущему снимку;
- pattern_clicked - клик по шаблону;
- reset_password - сброс пароля.

Таким образом, компоненты UI больше не связаны напрямую: кнопка не знает о сетке, сетка не знает о метке, а все взаимодействия проходят через UIMediator.

2. Описание паттерна «Снимок»

Паттерн «Снимок» (Memento) - это поведенческий паттерн проектирования, который позволяет сохранить и восстановить внутреннее состояние объекта, не нарушая его инкапсуляцию.

Данный паттерн применяется в ситуациях, когда нужно реализовать отмену действий, историю состояний или контрольные точки.

В разработанном приложении паттерн «Снимок» используется для отката состояния приложения к предыдущим шагам. В программе определены следующие роли:

- SequenceMemento - снимок, содержащий выбранную последовательность и режим;
- SequenceCaretaker - хранитель истории снимков (стек);
- AppWindow - объект-источник, который создаёт и восстанавливает состояние.

При каждом действии посредник создаёт новый снимок:

- memento = SequenceMemento(self.app.selected, self.app.mode)
- self.app.caretaker.save(memento)

При нажатии «↶ Отменить (снимок)» вызывается caretaker.undo(), который возвращает предыдущий снимок, и состояние восстанавливается:

- memento = self.app.caretaker.undo()
- selected, mode = memento.get_state()
- self.app.selected = selected
- self.app.mode = mode

Таким образом, пользователь может откатывать свои действия, а инкапсуляция состояния не нарушается.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются требования:

- длина от 4 до 9 ячеек;
- отсутствие повторов;
- ячейки в диапазоне 1–9.

Приложение содержит два режима: «Создание пароля» и «Вход».

Дополнительно реализованы:

- кнопка «↶ Отменить (снимок)» для отката к предыдущему состоянию;
- панель шаблонов («Крест», «Квадрат», «Зигзаг»);
- счётчик истории снимков внизу окна;
- кнопка «Очистить»;
- кнопка «Сбросить сохранённый пароль».

Все действия проходят через UIMediator, который сохраняет снимок и обновляет все компоненты.

4. Диаграмма классов

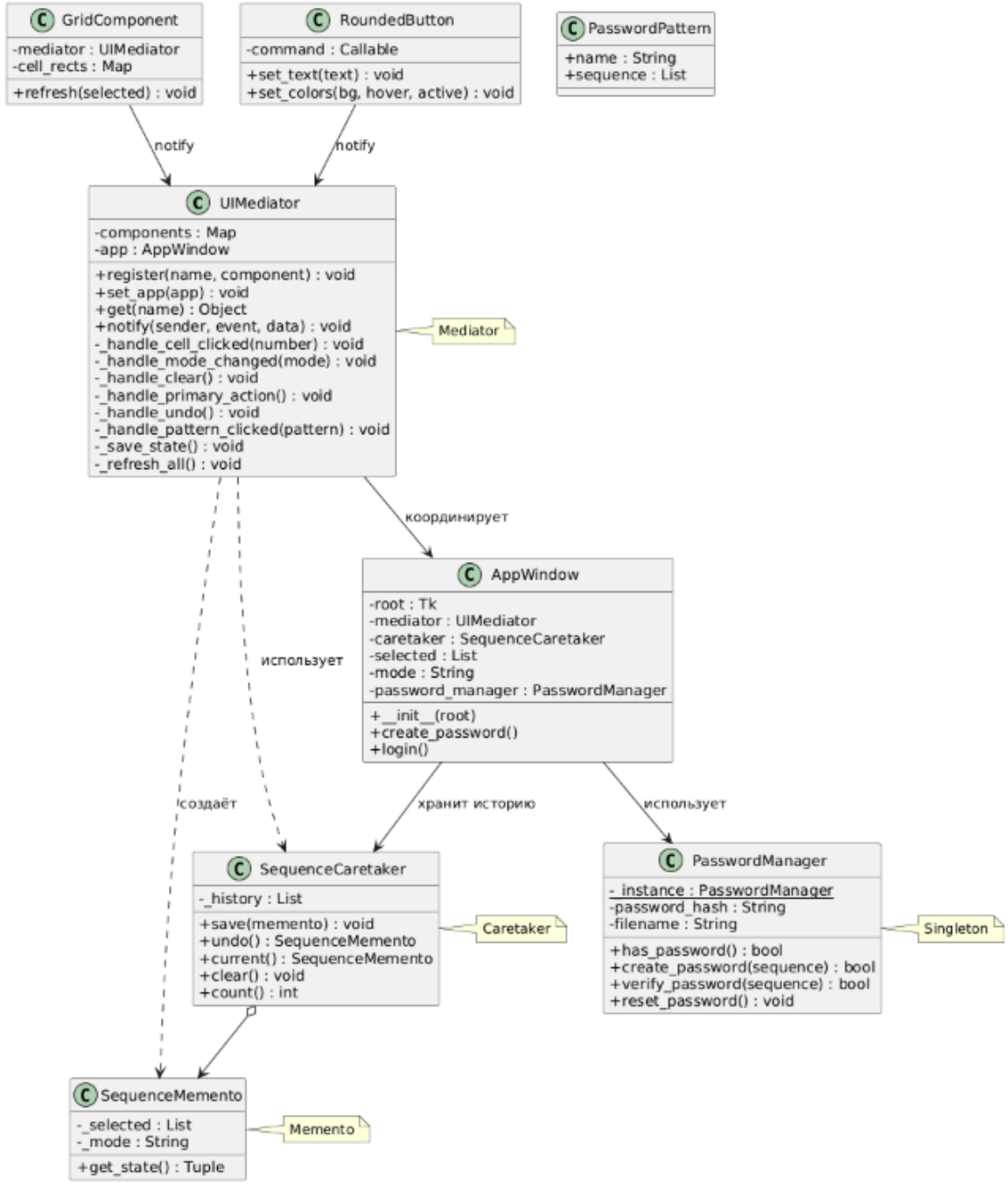


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит:

- UIMediator - посредник;
- GridComponent, RoundedButton - компоненты, общающиеся через посредника;
- SequenceMemento, SequenceCaretaker - снимок и хранитель;
- AppWindow, PasswordManager, PasswordPattern - основные классы.

5. Реализация паттерна «Посредник»

Посредник:

```
class UIMediator:
    def __init__(self):
        self.components = {}
        self.app = None

    def register(self, name, component):
        self.components[name] = component

    def set_app(self, app):
        self.app = app

    def notify(self, sender, event, data=None):
        if event == "cell_clicked":
            self._handle_cell_clicked(data)
        elif event == "mode_changed":
            self._handle_mode_changed(data)
        elif event == "clear":
            self._handle_clear()
        elif event == "primary_action":
            self._handle_primary_action()
        elif event == "undo":
            self._handle_undo()
        elif event == "pattern_clicked":
            self._handle_pattern_clicked(data)
        elif event == "reset_password":
            self._handle_reset_password()
```

Клик по ячейке в сетке:

```
self.tag_bind(tag, "<Button-1>",
```

```
            lambda e, num=i: self.mediator.notify(self, "cell_clicked", num))
```

Посредник обновляет все компоненты:

```
python
def _refresh_all(self):
    grid = self.get("grid")
    seq_label = self.get("sequence_label")
    primary_btn = self.get("primary_btn")
    ...
    grid.refresh(self.app.selected)
    seq_label.config(text=" → ".join(map(str, self.app.selected)))
    ...
```

Таким образом, компоненты не знают друг о друге - только о посреднике.

6. Реализация паттерна «Снимок»

Снимок:

```
class SequenceMemento:
    def __init__(self, selected, mode):
        self._selected = list(selected)
        self._mode = mode

    def get_state(self):
        return list(self._selected), self._mode
```

Хранитель:

```
class SequenceCaretaker:
    def __init__(self):
        self._history = []

    def save(self, memento):
        self._history.append(memento)
```

```
    def undo(self):
        if len(self._history) > 1:
            self._history.pop()
            return self._history[-1]
        if len(self._history) == 1:
            return self._history[0]
        return None

    def current(self):
        if not self._history:
            return None
        return self._history[-1]
```

Сохранение снимка при каждом действии:

```
def _save_state(self):
    memento = SequenceMemento(self.app.selected, self.app.mode)
    self.app.caretaker.save(memento)
```

Восстановление:

```
def _handle_undo(self):
    memento = self.app.caretaker.undo()
    if memento is None:
        self._set_status("Нечего отменять.", "#ffaa00")
        return
    selected, mode = memento.get_state()
    self.app.selected = selected
    self.app.mode = mode
    self._refresh_all()
    self._set_status("Действие отменено.", "#4a90d9")
```

Таким образом, паттерн «Снимок» позволяет откатывать действия пользователя, не нарушая инкапсуляцию AppWindow.

7. Результаты работы программы

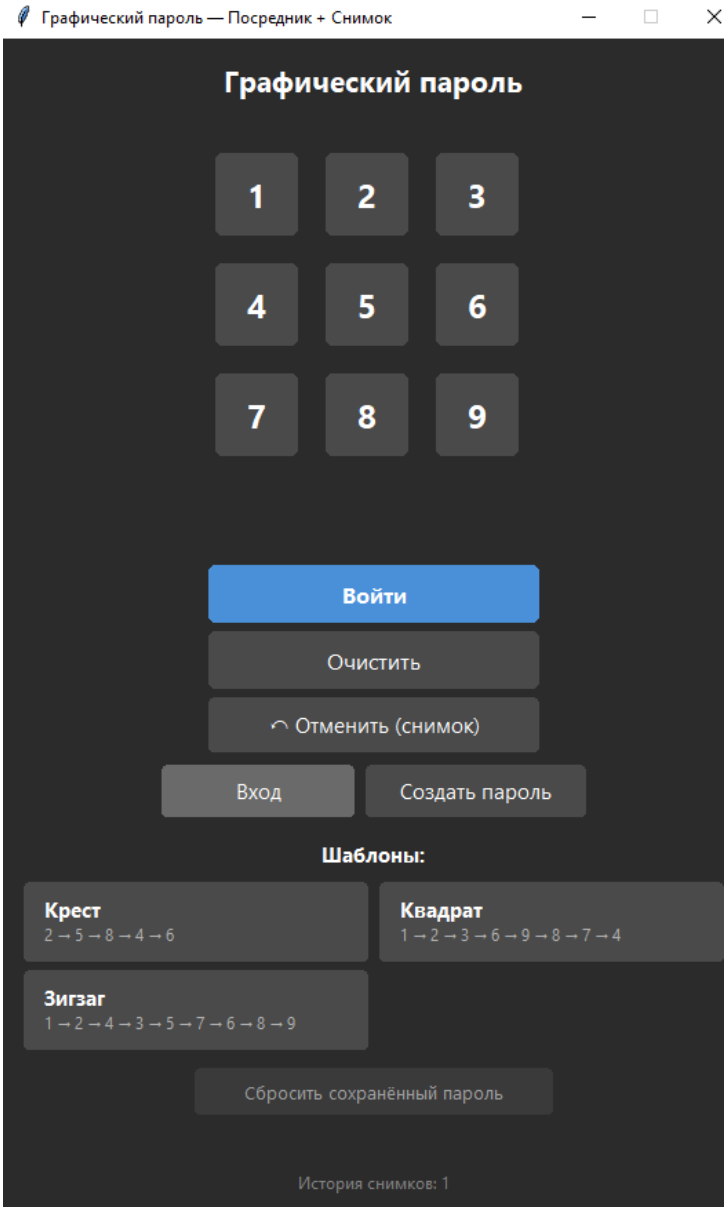


Рисунок 2 - Главное окно приложения

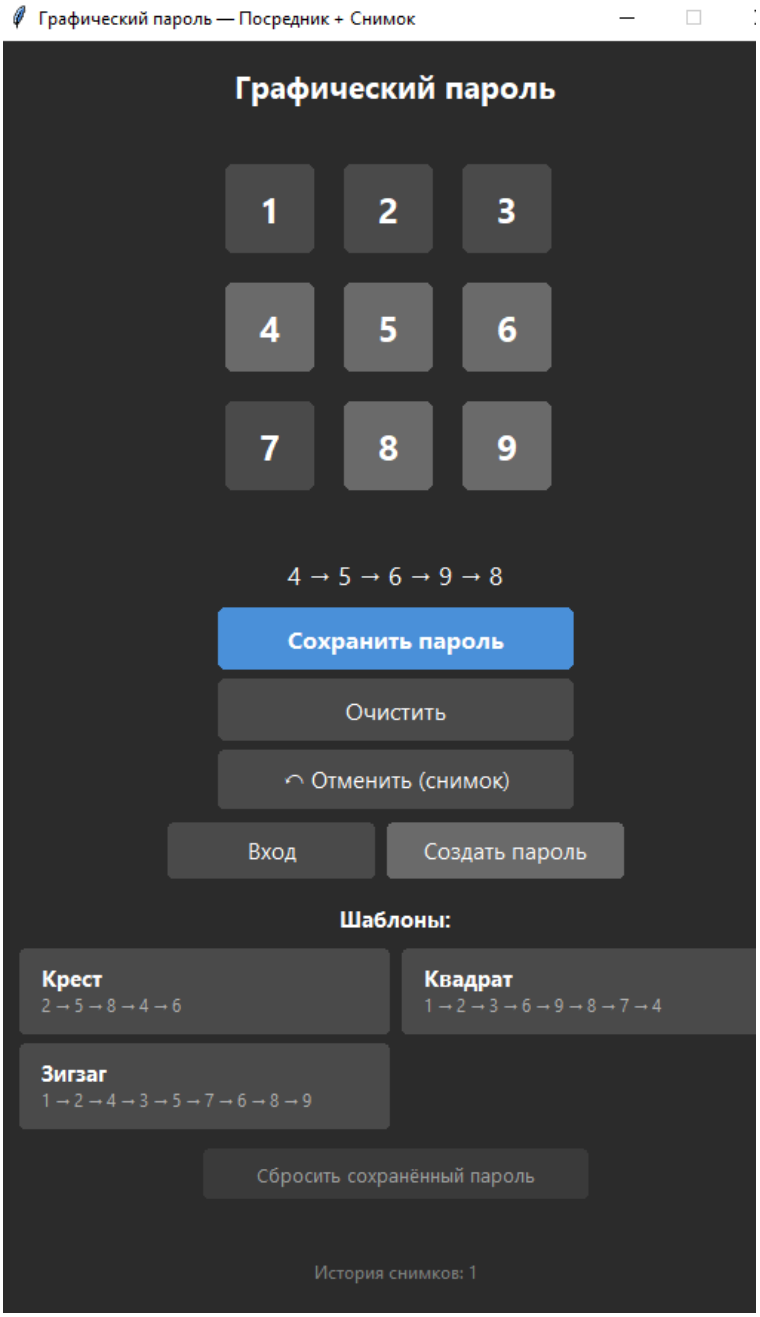


Рисунок 3 - Формирование последовательности (каждый клик создаёт снимок)

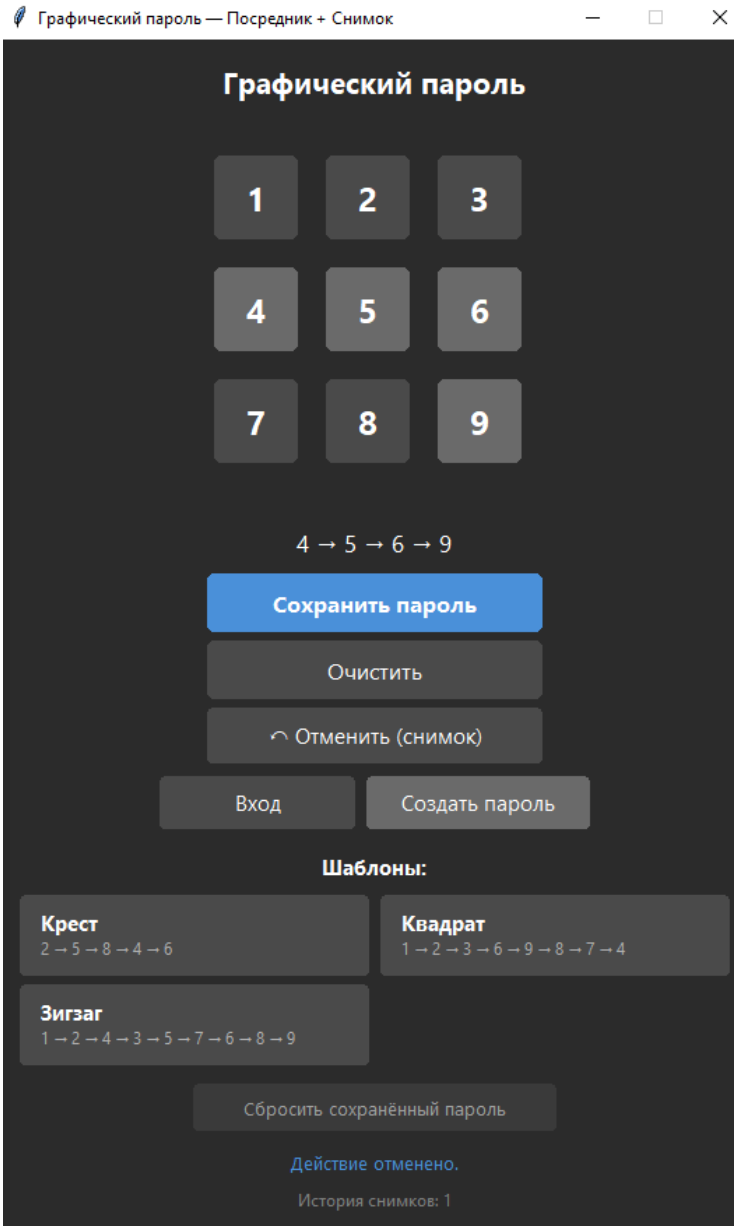


Рисунок 4 - Откат действия через снимок (↶ Отменить)

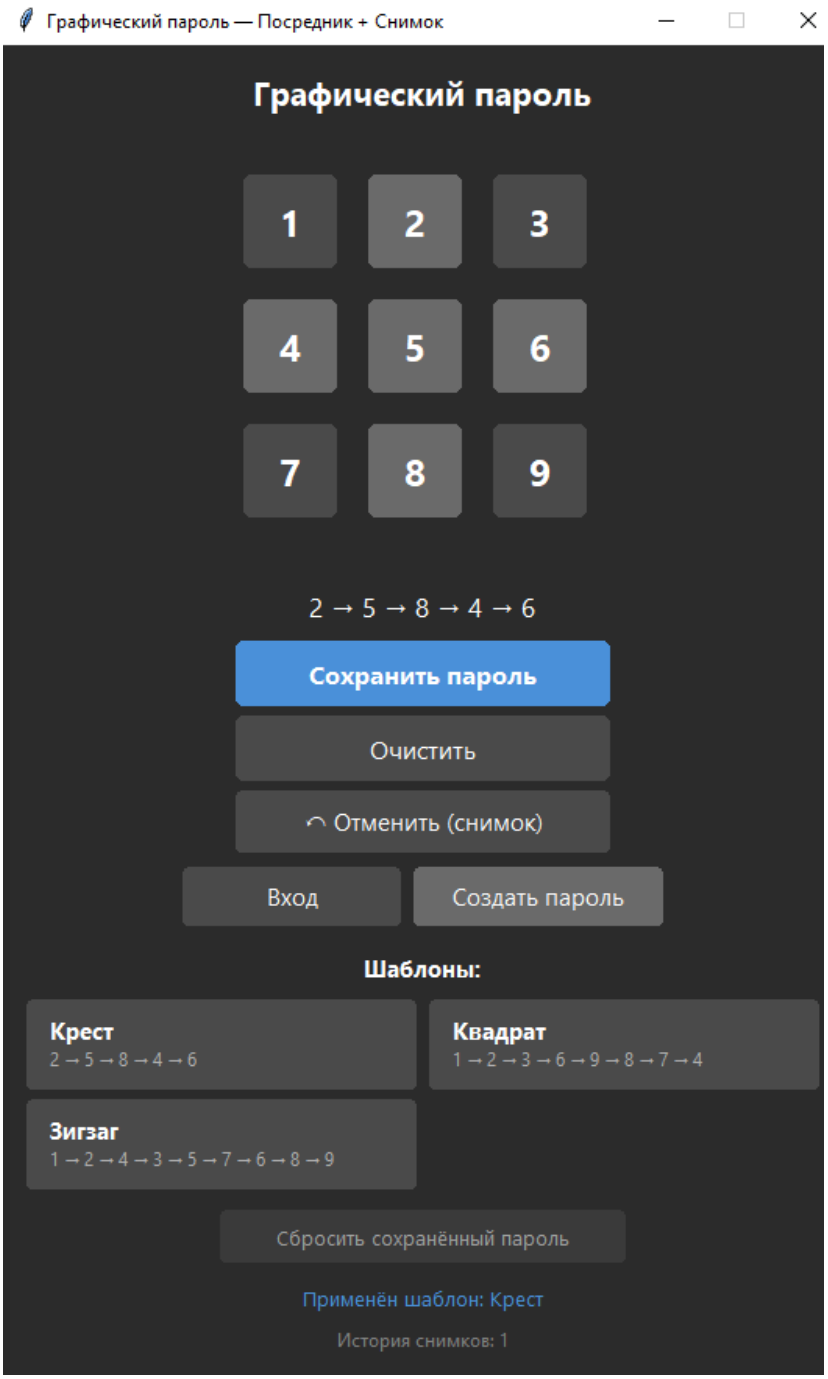


Рисунок 5 - Применение шаблона (тоже создаёт снимок)

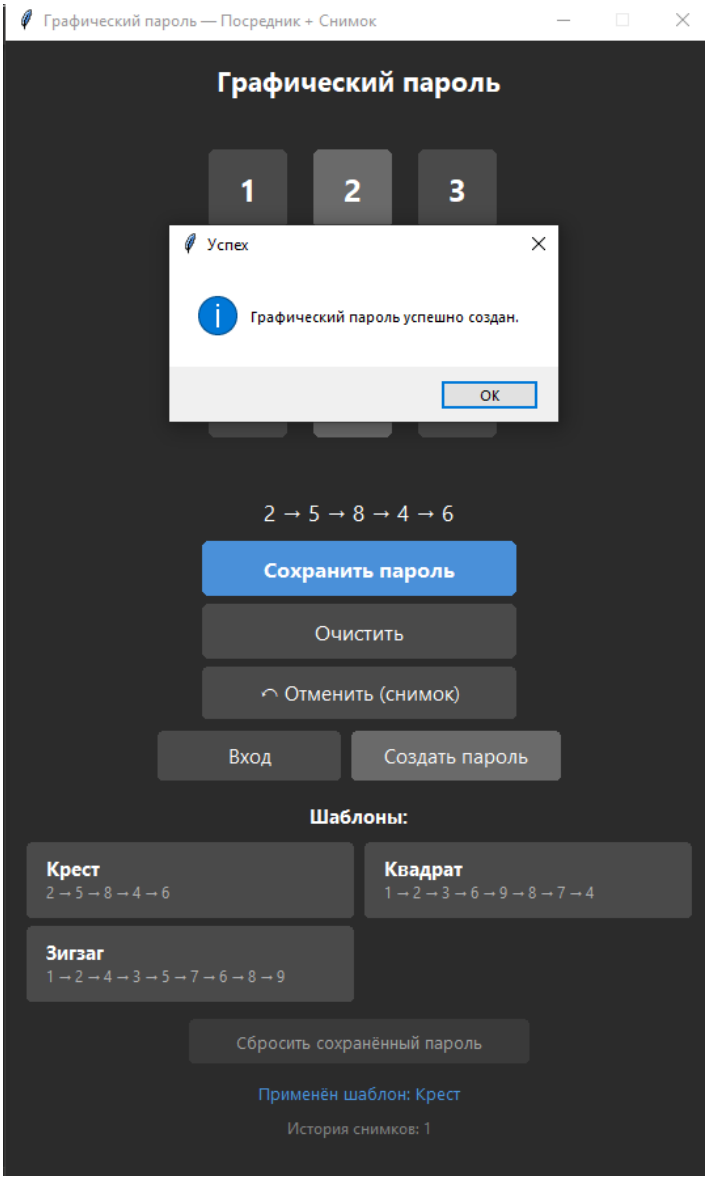


Рисунок 6 - Сохранение пароля

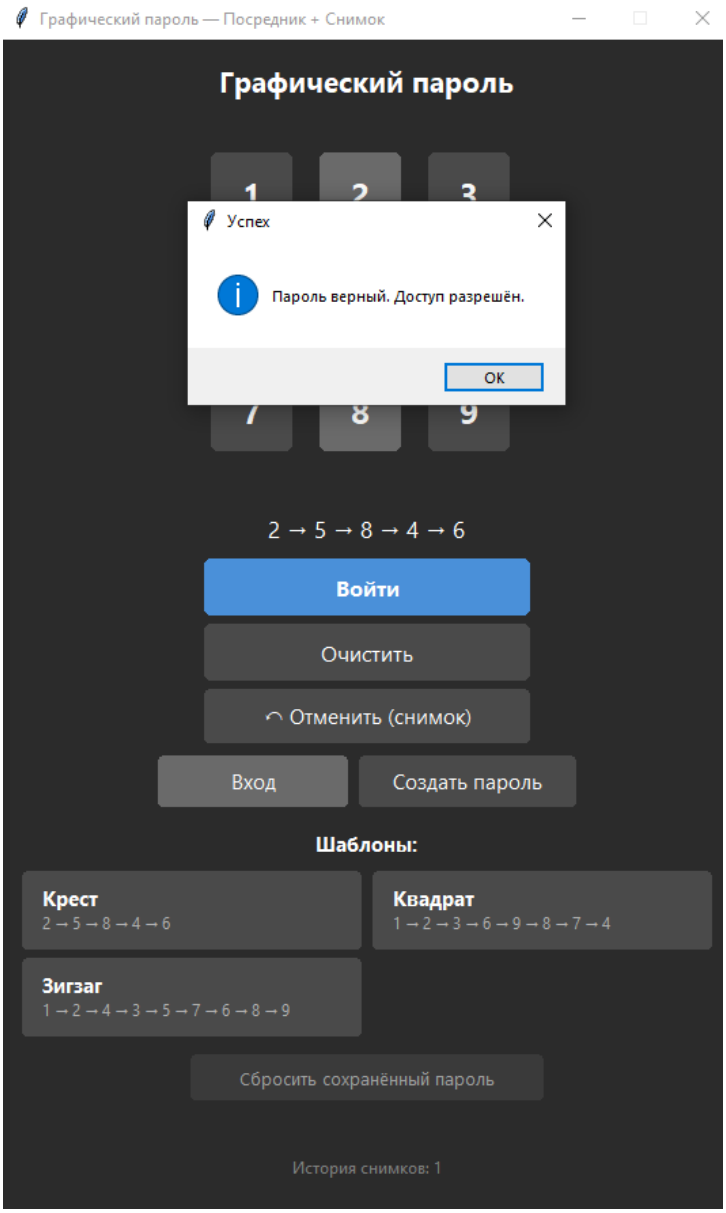


Рисунок 7 - Проверка правильного пароля

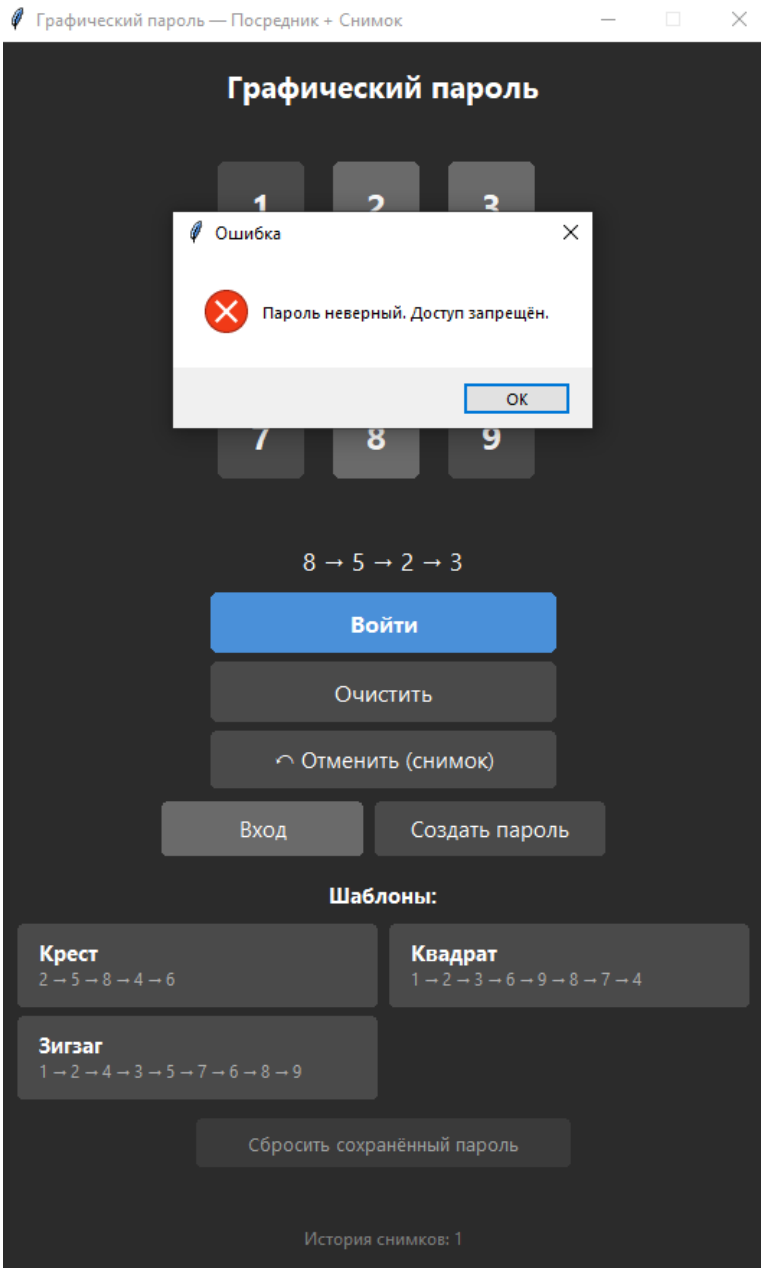


Рисунок 8 - Проверка неправильного пароля

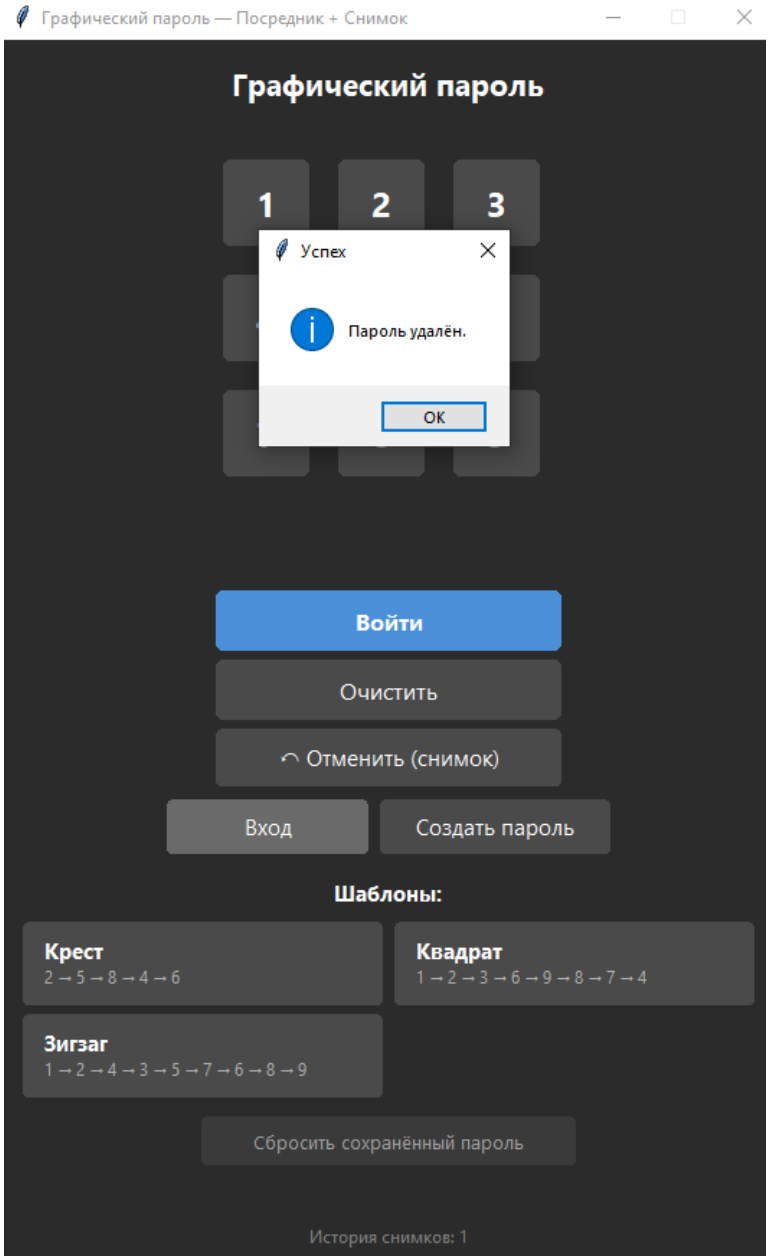


Рисунок 9 - Сброс сохранённого пароля

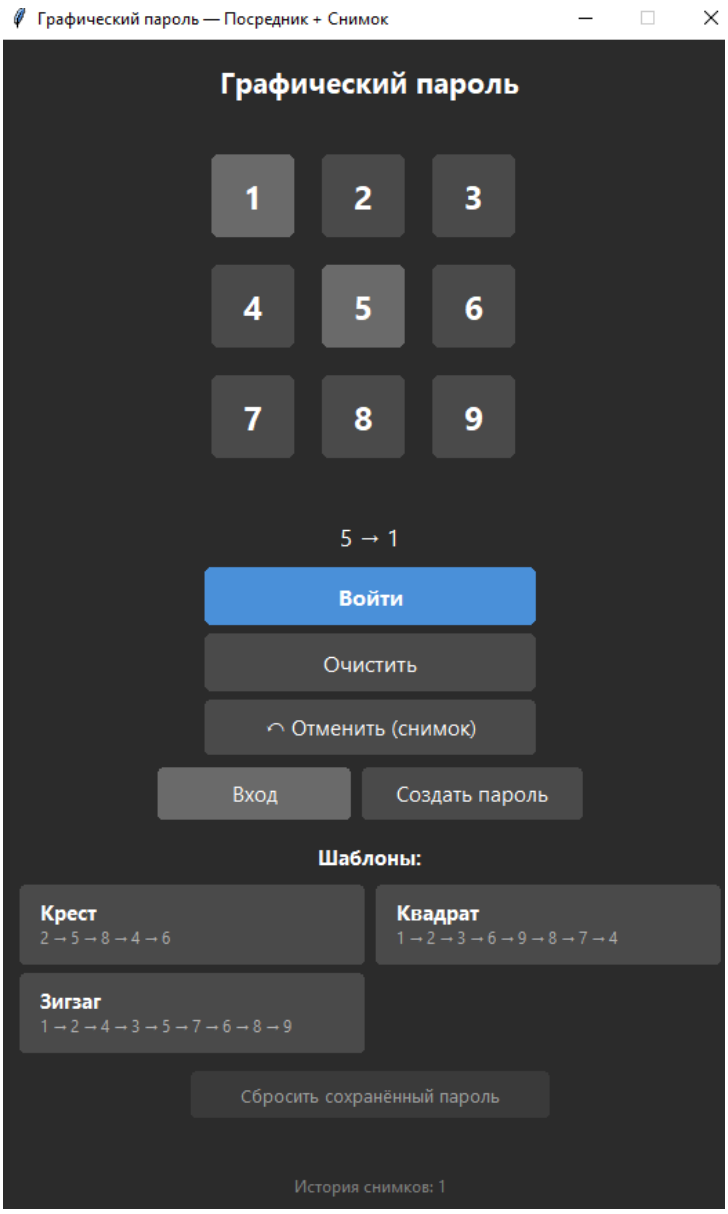


Рисунок 10 - Координация через посредника (визуально: клик по сетке обновляет метку и подсветку)

Работа посредника: клик по ячейке «5» одновременно подсветил её и обновил строку последовательности под сеткой. Сетка и метка не связаны напрямую - обновление координирует UIMediator

8. Исходный код

Основные элементы программы:

- UIMediator - посредник для UI-компонентов;
- GridComponent - сетка, общающаяся через посредника;
- SequenceMemento - снимок состояния;
- SequenceCaretaker - хранитель истории;

- PasswordPattern - шаблон пароля;
- PasswordManager - Singleton с хэшированием;
- AppWindow - основной класс приложения;
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;
- hash_sequence(), validate_sequence() - вспомогательные функции.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два поведенческих паттерна проектирования: «Посредник» (Mediator) и «Снимок» (Memento). Паттерн «Посредник» применён для организации взаимодействия UI-компонентов через UIMediator, что устранило прямые связи между сеткой, кнопками и метками. Паттерн «Снимок» применён для отката состояния: SequenceCaretaker хранит историю SequenceMemento, и пользователь может вернуться к предыдущему состоянию кнопкой «↶ Отменить». В результате получена работоспособная программа, демонстрирующая практическое применение поведенческих паттернов при разработке прикладного программного обеспечения.